
STL

Standard Template Library

Programación
Orientada a Objetos

Lic. Leonardo Brambilla

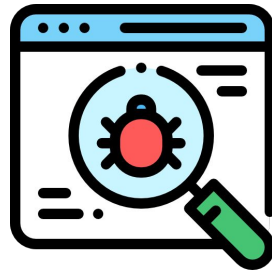
¿Por qué aprender STL?



Los errores son un componente inevitable del desarrollo de software.

"No tenemos que reinventar la rueda, solo tenemos que aprender a usarla mejor."

Cada línea de la STL que usamos es una línea que no tenemos que depurar.



¿Por qué aprender STL?



Es un conjunto de clases y funciones genéricas, implementadas como plantillas (templates), que facilitan el uso de estructuras de datos y algoritmos comunes sin que el programador tenga que re-implementarlos desde cero.

Ventajas de la STL

Código reutilizable: templates.

Eficiente: implementaciones muy optimizadas.

Consistente: misma interfaz para distintos contenedores.

Flexible: se puede usar con cualquier tipo de dato, incluso definidos por el usuario.



STL - Componentes



- **Contenedores:** estructuras de datos genéricas que almacenan elementos.

Cada uno tiene diferentes características de almacenamiento y eficiencia.

No saben de “algoritmos”, solo guardan y permiten acceso a los datos.

std::vector, std::list, std::deque, std::set, std::map

- **Iteradores:** Son Clases que permiten señalar datos o posiciones de los contenedores. Cada contenedor tiene sus propios iteradores, pero se usan de forma genérica.

Tipos comunes:

begin(), end()

Algunos soportan iteradores bidireccionales (*list*), otros aleatorios (*vector*).

- **Algoritmos:** Son funciones genéricas que operan sobre rangos de elementos usando iteradores. No dependen del contenedor, sino de los iteradores que se les pasen.

std::sort, std::find, std::count, std::for_each.

STL - Contenedores



1. Contenedores secuenciales: Implementan estructuras de datos con acceso secuencial.

Incluyen: vector list deque array forward_list

2. Adaptadores de contenedor: Implementan estructuras como colas y pilas proporcionando diferentes interfaces sobre los contenedores secuenciales.

Incluyen: stack queue priority_queue

3. Contenedores asociativos: Se usan para almacenar datos ordenados, que pueden buscarse rápidamente mediante una clave.

Incluyen: map multimap set multiset

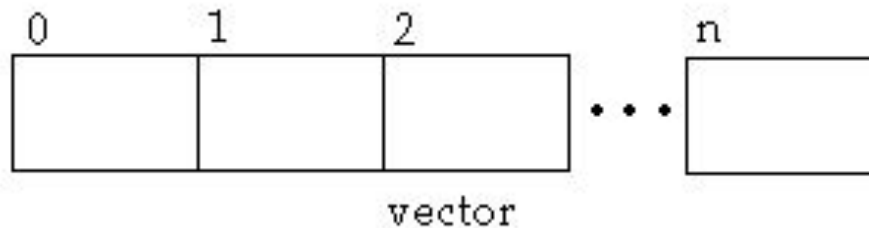
STL: vector



Arreglo dinámico con acceso al azar.

```
vector<int> v;  
v.resize(3);  
v[0]=1; v[1]=5; v[2]=7;  
v.push_back(10);  
v.insert(v.begin(), 1);  
v.insert(v.begin()+2,7);  
v.erase(v.begin()+2);  
v.pop_back();           // elimina el último  
for (int i=0;i<v.size();i++)  
    cout<<v[i]<<" ";
```

```
// crea un vector de enteros vacío  
// el vector ahora tendrá 3 elementos  
// asigna valores a esos 3 e.  
// agrega un 10 al final  
// Inserta al inicio  
// agrega un 7 en la 3ra pos.  
// elimina el 7 (3ra pos)
```

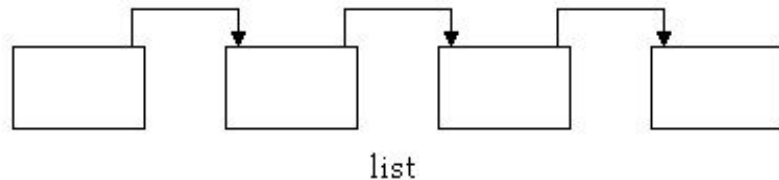


STL: list



Lista enlazada, acceso secuencial

```
list<int> l;           // crea una lista de enteros vacía
list<int>::iterator it; // iterador
l.push_back(10);       // agrega 10 al final
l.push_back(20);       // agrega 20 al final
l.push_front(5);       // agrega 5 al principio
it = l.begin();
it++;
it++;                 // apunta al 3er elemento
it = l.insert(it, 7);  // inserta 7 en la 3ra posición
it = l.erase(it);     // elimina el 7
for (it = l.begin(); it != l.end(); it++)
    cout << *it << " ";
```



STL: deque



Cola doblemente enlazada, acceso al azar

```
deque<int> d; // crea un deque de enteros vacío
```

```
// Insertamos elementos
```

```
d.push_back(10); // agrega un 10 al final
```

```
d.push_back(20); // agrega un 20 al final
```

```
d.push_front(5); // agrega un 5 al principio
```

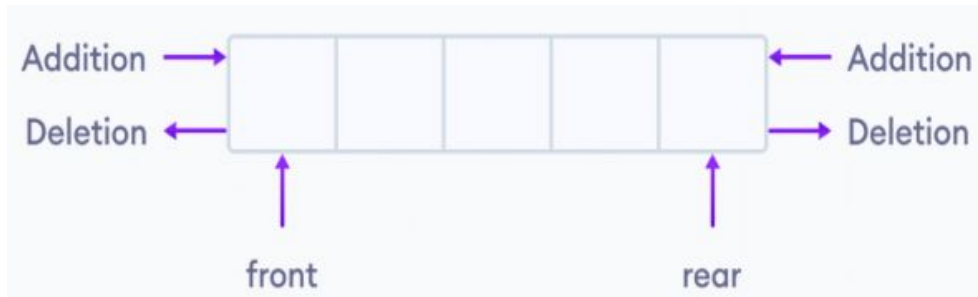
```
// Insertar en la 3ra posición (índice 2)
```

```
auto it = d.begin() + 2;
```

```
it = d.insert(it, 7); // agrega un 7 en la 3ra pos.
```

```
// Eliminar el elemento insertado
```

```
it = d.erase(it); // elimina el 7 (3ra pos.)
```



STL: map

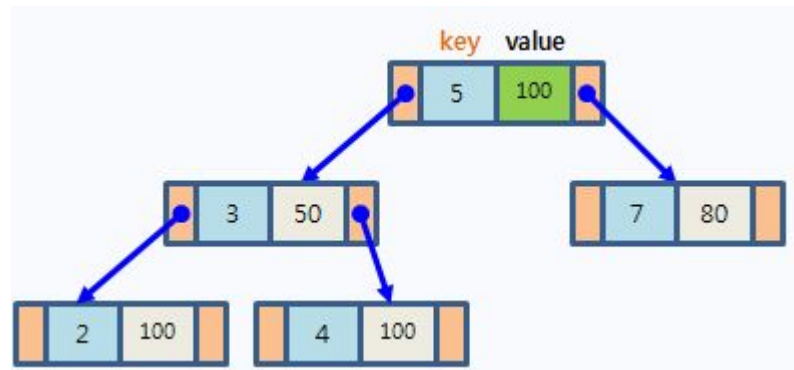


Un map es un contenedor asociativo ya que dado un valor (key), encontrará el otro valor (value):

```
map<string, int>    //es un mapeo de strings a int  
map<int, int>       //es un mapeo de int a int
```

maps son 1:1

multimap son 1:n



STL: map



```
map<string,int> calif;  
calif["Fundamentos de Programación"]=9;  
calif["P00"]=7;  
calif["Matemática Básica"]=8;  
calif["Física I"]=7;  
calif["Álgebra"]=7;  
calif["Ingeniería de Software"]=7;  
string materia;  
getline(cin,materia);  
if (calif.find(materia)==calif.end())  
    cout<<"No se registra nota para esa materia.";  
else  
    cout<<"La nota es: "<<calif[materia];
```



STL: <algorithm>

Colección de algoritmos genéricos que operan sobre rangos (como los de vector, list, array, etc.)

```
bool es_par(int x) {  
    return x%2==0;  
};  
  
list<int> l;  
list<int>::iterator it;  
  
// busca el valor 10 en la lista  
it = find( l.begin() , l.end() , 10 );  
  
// busca el primer número par  
it = find_if( l.begin() , l.end() , es_par );
```

STL: <algorithm>



```
std::list<int> l = {10, 2, 10, 3, 4, 10, 5};  
  
// elimina todas las veces que aparece el valor 10  
std::remove(l.begin(), l.end(), 10);                                //-> [2, 3, 4, 5, ?, ?, ?]  
  
// l.size() sigue siendo 7 - El rango útil (sin los "10") es {2, 3, 4, 5, ...}  
l.erase(std::remove(l.begin(), l.end(), 10), l.end());              //-> [2, 3, 4, 5]  
  
// elimina todos los números pares de la lista  
remove_if( l.begin() , l.end() , es_par );  
  
// pares:  
l.erase(std::remove_if(l.begin(), l.end(), es_par), l.end());  
  
// reemplaza todos los 15 por 30  
replace( l.begin() , l.end() , 15 , 30 );  
  
// reemplaza todos los pares por 0  
replace_if( l.begin() , l.end() , es_par , 0 );
```

STL: <algorithm>



// cuenta cuantas veces aparece el valor 2

```
int n1 = count( l.begin() , l.end() , 2);
```

// cuenta cuántos números pares hay en la lista

```
int n2 = count_if( l.begin() , l.end() , es_par );
```

// suma todo el contenido de la lista

```
n = accumulate( l.begin() , l.end() , 0 );
```

// elimina los repetidos de la lista

```
l.sort();
```

// ordena la lista

```
it = unique(l.begin(), l.end());
```

// agrupa los repetidos

```
l.erase(it, l.end());
```

// elimina los duplicados

-> [1, 2, 1, 3, 2]

-> [1, 2, 1, 3, 2] = 2

-> n2 = 9

-> n = 9

-> [1, 2, 1, 3, 2]

-> [1, 1, 2, 2, 3]

-> [1, 2, 3, 2, 3]

-> [1, 2, 3]

STL: <algorithm>



// busca el maximo y minimo

```
it = max_element( l.begin() , l.end() );
```

//-> [1, 2, 1, 3, 2]

```
it = min_element( l.begin() , l.end() );
```

//-> [1, 2, 1, 3, 2]

//it no contiene el número, sino que apunta al número.

```
cout << *it;
```

// desordena el vector

```
std::shuffle( v.begin() , v.end() );
```

// invierte el orden de la lista

```
reverse( l.begin() , l.end() );
```

//-> [2, 3, 1, 2, 1]